THE PROVING SYSTEM OF ZISK

ZISK CORE TEAM

Abstract. TODO.

Contents

1. Introduction **1**
2. Multilinear Arithmetization **2**
3. Commitment Scheme **4**
4. The Zerocheck PIOP **6**
5. Opening the Committed Columns **7**
6. The Basic-Proof Protocol **8**
7. From AIRs to Circuits **10**
8. Instantiation and Scope **11**
References **13**

1. Introduction

A proving system separates cleanly into a *frontend*, which turns a computation into a system of polynomial constraints, and a *backend*, which turns a witness for those constraints into a succinct cryptographic proof. The ZisK whitepaper (its Arithmetization Framework section) specifies the frontend in full: the AIR framework, interactions, buses, and the LogUp reduction of bus balancing. It treats the backend abstractly —a one-time setup produces a proving key pk and a verification key vk, and verification of a proof π against a public statement x is a deterministic procedure $V(\mathsf{vk}, x, \pi) \in \{\mathsf{accept}, \mathsf{reject}\}$— without committing to how π is produced.

This document fills that gap with a concrete backend: a multilinear STARK. Where a classical STARK [Ben+18] views each trace column as a univariate polynomial over a multiplicative subgroup and enforces constraints by low-degree testing a quotient, the multilinear approach views each column as a function on the Boolean hypercube, enforces constraints by a *sumcheck* [Lun+92], and commits columns with the Basefold polynomial commitment scheme [ZCF23]. The two layers remain decoupled: the same frontend AIRs are proved unchanged; only the backend differs.

1.1. What a Basic Proof Is.

The ZisK circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$ is proved piecewise. Each *instance* is a concrete witness for one of the AIRs $(\mathcal{A}_\tau, \mathcal{I}_\tau) \in \mathcal{C}$, and a *basic proof* is the backend's proof for a single instance. In the language of the whitepaper's recursion layer, a basic proof is exactly a proof for the AIR-leaf relation

$\mathcal{R}^{(\tau)}_{\text{AIR}}$: given a challenge $\alpha \in \mathbb{K}$, the program publics $\boldsymbol{p}$, and a claimed bus contribution s, it attests that there is a witness $(\boldsymbol{T}, \boldsymbol{a})$ that

1. satisfies the AIR $\mathcal{A}_\tau$ and is consistent with $\boldsymbol{p}$, and
2. produces the partial bus sum s under α.

Basic proofs are the leaves of the aggregation tree; combining them into a single circuit-level statement —checking global bus balancing $\sum_i s_i = 0$ under a common challenge— is the job of recursion (the whitepaper's Recursion section) and is out of scope here. What this document specifies is precisely how one basic proof is generated and verified, plus the way a *set* of basic proofs shares a challenge and exposes the data the aggregation layer consumes (Section 7).

1.2. **Preliminaries and Notation.**

We work over the Goldilocks prime field $\mathbb{F}$ with $p = 2^{64} - 2^{32} + 1$, and its degree-three extension $\mathbb{K} = \mathbb{F}[X]/(X^3 - X - 1)$, used for all soundness-critical randomness. We reuse the whitepaper's AIR notation without change: an AIR $\mathcal{A} = \{C_i\}_{i \in I}$ of width W, depth D, and V values; a witness $(\boldsymbol{T}, \boldsymbol{a})$ with trace matrix $\boldsymbol{T} \in \mathbb{F}^{N \times W}$ and AIR values $\boldsymbol{a} \in \mathbb{F}^V$; columns $T_j : [N] \to \mathbb{F}$; the shifted variable $X_j^{(k)}$ instantiated at row r by $T_{j(r+k)}$ (indices modulo N); the value variable Y_j instantiated by a_j. Throughout we take the trace height to be a power of two, $N = 2^n$, and we identify a row index $r \in [N]$ with its little-endian bit decomposition

$$\boldsymbol{r} = (r_0, r_1, ..., r_{n-1}) \in \{0,1\}^n, \quad r = \sum_{i=0}^{n-1} r_i 2^i.$$

Vectors are set in bold; $\boldsymbol{p} \in \mathbb{F}^k$ is the public statement.

2. Multilinear Arithmetization

The single structural change from a univariate STARK is how the trace is read. Instead of interpolating each column on a multiplicative subgroup, we regard it as a function on the Boolean hypercube and take its multilinear extension.

2.1. **Columns as Multilinear Polynomials.**

Definition 1 (Multilinear Extension of a Column). Let $T_j : [N] \to \mathbb{F}$ be a trace column with $N = 2^n$. Its *multilinear extension* (MLE) is the unique polynomial $\tilde{T}_j \in \mathbb{F}[X_1, ..., X_n]$ of individual degree ≤ 1 in each variable such that

$$\tilde{T}_j(\boldsymbol{b}) = T_j(r_{\boldsymbol{b}}) \quad \text{for every } b \in \{0,1\}^n,$$

where $r_{\boldsymbol{b}}$ is the row index whose bit decomposition is $\boldsymbol{b}$.

It is given explicitly by

$$\tilde{T}_j(\boldsymbol{X}) = \sum_{\boldsymbol{b} \in \{0,1\}^n} T_j(r_{\boldsymbol{b}}) \cdot \mathrm{eq}(\boldsymbol{b}, \boldsymbol{X}),$$

where eq is the *equality kernel*, the multilinear indicator of $\boldsymbol{X} = \boldsymbol{Y}$ on the hypercube; that is:

$$\mathrm{eq}(\boldsymbol{X}, \boldsymbol{Y}) = \prod_{i=1}^{n} [X_i Y_i + (1 - X_i)(1 - Y_i)],$$

The map $T_j \mapsto \tilde{T}_j$ is a bijection between columns (functions $[N] \to \mathbb{F}$) and n -variate multilinear polynomials: both spaces have dimension $N = 2^n$. This is the multilinear analogue of "a degree-$(< N)$ univariate is determined by its values on N points", and it is the only sense in which the trace is reinterpreted; the AIR and its constraints are untouched.

2.2. Shifted Columns and the Rotation Kernel.

An AIR of depth D references not only $T_j(r)$ but the shifted values $T_j(r + k)$ for $0 \leq k \leq D$, with the row index taken modulo N. The shifted column $T_j^{(k)} : r \mapsto T_j(r + k)$ has its own MLE $\tilde{T}_j^{(k)}$, but it need not be committed separately: it is a fixed linear image of $\tilde{T}_j$.

Definition 2 (Rotation Kernel). For an offset k, let $\varphi_k : \{0,1\}^n \to \{0,1\}^n$ be the bijection that increments a bit-address by k modulo 2^n (so $\varphi_k(\boldsymbol{r}) = \boldsymbol{r} + \boldsymbol{k}$). The *rotation kernel* is the bivariate polynomial, multilinear in $\boldsymbol{Y}$,

$$\mathrm{rot}_k(\boldsymbol{x}, \boldsymbol{Y}) = \mathrm{eq}(\boldsymbol{x}, \varphi_k(\boldsymbol{Y})),$$

and it satisfies, for every column T_j,

$$\tilde{T}_j^{(k)}(\boldsymbol{Y}) = \sum_{\boldsymbol{x} \in \{0,1\}^n} \tilde{T}_j(\boldsymbol{x}) \cdot \mathrm{rot}_k(\boldsymbol{x}, \boldsymbol{Y}).$$

Incrementing by k ripples a binary carry, so rot_k is eq composed with that carry chain; for $k = 0$ it degenerates to eq.

Because the increment is a carry ripple rather than a clean coordinate map, evaluating $\mathrm{rot}_k(\cdot, \boldsymbol{z})$ at a point $\boldsymbol{z}$ is not a single coordinate substitution; it

is computed in $O(n)$ by a two-state dynamic program over the carry bit. The consequence is what matters: a depth-D AIR commits only its W base columns, and every shifted reference $T_j(r+k)$ becomes an evaluation of the *same* committed $\tilde{T}_j$ against a verifier-computable kernel.

2.3. **Column Categories and AIR Values.**

The whitepaper's three column categories map onto the backend as follows. *Fixed* columns are committed once at setup and summarized in vk. *Cached* columns (e.g. the ROM instruction trace) are committed once and reused; the backend treats each cached commitment as an additional fixed matrix whose root is bound into the transcript. *Witness* columns are committed per instance. The value vector $\boldsymbol{a} \in \mathbb{F}^V$ is not indexed by row; each a_j enters constraints as the constant Y_j, and—crucially for the circuit-level argument—the components of $\boldsymbol{a}$ that a stage ≥ 2 argument produces (notably a partial bus sum s) are exposed as part of the instance's public output.

2.4. **Constraint Satisfaction as Vanishing on the Hypercube.**

Substituting the MLEs into a constraint gives a polynomial in $\boldsymbol{X}$. Writing $\tilde{\boldsymbol{T}}(\boldsymbol{X})$ for the tuple of all base and shifted column MLEs evaluated at $\boldsymbol{X}$, define the *composed constraint*

$$\hat{C}_i(\boldsymbol{X}) = C_i\big(\tilde{\boldsymbol{T}}(\boldsymbol{X}), a_1, ..., a_V\big).$$

Its individual degree may exceed 1 (a multiplication gate multiplies columns), so it is generally not multilinear. The witness satisfies $\mathcal{A}$ if and only if every composed constraint vanishes at every hypercube point:

$$\hat{C}_i(\boldsymbol{b}) = 0 \quad \text{for all } \boldsymbol{b} \in \{0,1\}^n \text{ and all } i \in I.$$

This is $|I| \cdot N$ scalar conditions. The zerocheck PIOP (Section 4) collapses them into a single sumcheck.

Remark. *There is deliberately* no *quotient here. Univariately, "vanishes on the domain H" is equivalent to divisibility by the sparse vanishing polynomial $Z_{H(X)} = X^N - 1$, and the backend commits and low-degree-tests the quotient C/Z_H. On the hypercube the ideal of polynomials vanishing on $\{0,1\}^n$ is generated by $\{X_i^2 - X_i\}$ and is not principal: there is no single Z_H to divide by. The multilinear backend therefore replaces "quotient and low-degree-test" with "multiply by* $\mathrm{eq}(\boldsymbol{r}, \cdot)$ *and sumcheck to zero". A direct consequence is that constraint degree costs the prover sumcheck work but never forces a larger committed object, unlike the univariate quotient whose degree is bounded by the code rate [Che+22].*

3. Commitment Scheme

The prover commits each column with Basefold [ZCF23], a polynomial commitment scheme for multilinears built from a foldable Reed–Solomon

code. We fix a rate parameter $\kappa \geq 1$ (the log-blowup) and write $\rho = 2^{-\kappa}$ for the code rate.

3.1. The Univariate–Multilinear Bijection.

Basefold commits a multilinear by committing an equivalent univariate. Order the 2^n multilinear monomial coefficients of $\tilde{T}_j$ into a vector $\boldsymbol{c} \in \mathbb{F}^{2^n}$ (the Möbius transform of the column values), and form the univariate

$$\hat{g}_j(X) = \sum_{i=0}^{2^n-1} c_i X^i, \quad \text{so that} \quad \hat{g}_j(x) = \tilde{T}_j\big(x^{2^0}, x^{2^1}, ..., x^{2^{n-1}}\big).$$

This is a bijection between n-variate multilinears and univariates of degree $< 2^n$. The point of using the *coefficient* form is the folding identity below.

3.2. Encoding and Merkle Commitment.

Definition 1 (Column Commitment). Let g be a coset generator and H a multiplicative subgroup with $|H| = 2^{n+\kappa}$. The *Reed–Solomon codeword* of column T_j is the evaluation vector $\big(\hat{g}_j(g \cdot \omega)\big)_{\omega \in H} \in \mathbb{F}^{2^{n+\kappa}}$. The *commitment* to T_j is the Merkle root of this codeword. A stage (a group of columns committed together) is committed by one Merkle tree whose leaf at position ℓ holds the values of all the stage's codewords at the paired points ℓ and $\ell + |H|/2$.

Pairing ℓ with $\ell + |H|/2$ per leaf is deliberate: these two positions map to x and $-x$, which is exactly the pair a folding-consistency check reads, so a query opens one leaf per tree. The verification key vk is the tuple of commitments to the fixed and cached columns; per-instance witness stages are committed by the prover and their roots travel in the proof.

3.3. Folding is Partial Evaluation.

The mechanism that makes Basefold an *evaluation* scheme, and not merely a low-degree test, is that one folding step of the codeword equals binding one variable of the multilinear.

Definition 2 (Fold). For $\lambda \in \mathbb{K}$, the *fold* of a codeword f on the domain gH maps $gH \to (gH)^2$ (half the size) by

$$\mathrm{Fold}(f, \lambda)(x^2) = \frac{f(x) + f(-x)}{2} + \lambda \cdot \frac{f(x) - f(-x)}{2x}.$$

If f is the codeword of $\tilde{T}_j$, then $\mathrm{Fold}(f, \lambda)$ is the codeword of $\tilde{T}_j(\lambda, X_2, ..., X_n)$: folding with λ fixes the first variable to λ. Equivalently, in the coefficient form, folding is $c'_i = c_{2i} + \lambda \cdot c_{2i+1}$. Iterating n folds with challenges $\lambda_1, ..., \lambda_n$ collapses the codeword to the single scalar $\tilde{T}_j(\lambda_1, ..., \lambda_n)$. This is the identity every multilinear PCS in the Basefold/

WHIR [Arn+24] family turns on, and it is what lets an evaluation proof be produced by folding rather than by a separate opening argument.

4. The Zerocheck PIOP

The zerocheck is the polynomial interactive oracle proof (PIOP) that certifies the $|I| \cdot N$ pointwise conditions of Section 2 with a single sumcheck. It is the multilinear counterpart of the univariate quotient check.

4.1. Batching Constraints and the eq Trick.

Two independent reductions collapse the conditions. First, all constraints are batched with a challenge $\mu \in \mathbb{K}$ into

$$C_\mu(\boldsymbol{X}) = \sum_{i \in I} \mu^{i-1} \hat{C}_i(\boldsymbol{X}),$$

sampled after the trace is committed. Batching does not raise the degree: $\deg C_\mu = \max_i \deg \hat{C}_i$. Second, a single constraint vanishes on the whole cube if and only if its multilinear extension is the zero polynomial, which is caught at a random point:

Definition 1 (Zerocheck). For a challenge point $\boldsymbol{r} \in \mathbb{K}^n$, define

$$G(\boldsymbol{X}) = \mathrm{eq}(\boldsymbol{r}, \boldsymbol{X}) \cdot C_\mu(\boldsymbol{X}).$$

Then $\hat{C}_i(\boldsymbol{b}) = 0$ for all $\boldsymbol{b} \in \{0,1\}^n$ and all i implies $\sum_{\boldsymbol{b} \in \{0,1\}^n} G(\boldsymbol{b}) = 0$; conversely, if some constraint is violated, then over the random draw of $(\boldsymbol{r}, \mu)$ the sum is non-zero except with probability $O((|I| + n)/|\mathbb{K}|)$ by Schwartz–Zippel.

The quantity $\sum_{\boldsymbol{b}} G(\boldsymbol{b})$ is precisely the multilinear extension of $C_\mu|_{\{\text{cube}\}}$ evaluated at $\boldsymbol{r}$, which is why a single random $\boldsymbol{r}$ suffices. Both $\boldsymbol{r}$ and μ are drawn, by Fiat–Shamir [FS87], after all commitments.

4.2. Running the Sumcheck.

The sumcheck [Lun+92] proves $\sum_{\boldsymbol{b}} G(\boldsymbol{b}) = 0$ in n rounds. In round t, with challenges $\lambda_1, ..., \lambda_{t-1}$ fixed, the prover sends the univariate round polynomial

$$g_t(Z) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-t}} G(\lambda_1, ..., \lambda_{t-1}, Z, \boldsymbol{b}),$$

of degree $d = \deg C_\mu + 1$ (the $+1$ is the eq factor). It is transmitted as its evaluations at $Z = 0, 1, ..., d$. The verifier checks $g_t(0) + g_t(1) = s_{t-1}$ (with $s_0 = 0$ and $s_t = g_t(\lambda_t)$) and replies with a fresh $\lambda_t \in \mathbb{K}$. After n rounds the claim is the single value $G(\boldsymbol{\lambda})$ at $\boldsymbol{\lambda} = (\lambda_1, ..., \lambda_n)$.

The prover evaluates the round polynomials by running the AIR's constraint expression over the partially bound column tables; the round degree, and hence prover cost per row, is set by the highest-degree gate.

Because the composed constraint is evaluated at $d+1$ points per pair rather than committed, a high-degree gate costs field work linear in d and never a larger commitment (this is the multilinear analogue of high-degree custom gates being cheap [Che+22]; further round-polynomial savings are possible [Gru24]).

4.3. **From the Final Claim to Evaluation Claims.**

To discharge $G(\boldsymbol{\lambda})$ the verifier needs to reconstruct it. It computes $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{\lambda})$ itself and knows μ and how each C_i is built from the columns, so it can assemble $G(\boldsymbol{\lambda}) = \mathrm{eq}(\boldsymbol{r}, \boldsymbol{\lambda}) \cdot \sum_i \mu^{i-1} C_{i(\ldots)}$ once it is told each *committed column value the constraints reference at* $\boldsymbol{\lambda}$. These are the evaluation claims:

$$\underbrace{\tilde{T}_1(\boldsymbol{\lambda}), ..., \tilde{T}_W(\boldsymbol{\lambda})}_{\text{current-row openings}} \quad \underbrace{\tilde{T}_j^{(k)}(\boldsymbol{\lambda}) \ \text{ for each } \ j, k.}_{\text{shifted-row openings, via } \mathrm{rot}_k}$$

By the rotation identity each shifted evaluation reduces to an evaluation of the *base* column $\tilde{T}_j$ against the kernel $\mathrm{rot}_k(\cdot, \boldsymbol{\lambda})$. No matter how many constraints or offsets an AIR has, the backend reduces to a set of "open this committed column at this point" claims against the W base columns only.

4.4. **Boundary Constraints.**

Constraints that must hold only on the first or last row (the whitepaper's boundary conditions, e.g. binding a public to row 0) never enter the sumcheck. A first-row constraint reduces to an evaluation claim at the hypercube corner $\mathbf{0}$ and a last-row constraint to the corner $\mathbf{1} = (1, ..., 1)$; the verifier evaluates the constraint on the claimed corner openings directly. In the opening (Section 5) these corner claims are handled by the equality kernel $\mathrm{eq}(\cdot, \mathbf{0})$ (resp. $\mathrm{eq}(\cdot, \mathbf{1})$), a special case of the machinery already in place.

5. Opening the Committed Columns

The zerocheck leaves a set of evaluation claims on the committed columns. Every such claim —plain, shifted, or corner— is a weighted sum over the hypercube against a verifier-computable kernel, and Basefold discharges them all with one batched opening.

5.1. **Claims as Weighted Sums.**

Each claim has the form $v = \sum_{\boldsymbol{b} \in \{0,1\}^n} w_j(\boldsymbol{b}) \cdot K(\boldsymbol{b})$, where w_j is a committed column and K is a *kernel* the verifier can evaluate at any point:

- a plain opening $\tilde{w}_j(\boldsymbol{\lambda})$ uses $K = \mathrm{eq}(\cdot, \boldsymbol{\lambda})$;
- a shifted opening $\tilde{w}_j^{(k)}(\boldsymbol{\lambda})$ uses the rotation kernel $K = \mathrm{rot}_k(\cdot, \boldsymbol{\lambda})$;
- a corner claim $w_j(r)$ at a boundary row uses $K = \mathrm{eq}(\cdot, \boldsymbol{r})$.

Enumerate the committed columns $w_1, ..., w_L$ (witness stages, then fixed, then cached) and the distinct kernels $K_1, ..., K_m$. The prover sends the full matrix of claimed values $v_{j,i} = \sum_{\boldsymbol{b}} w_j(\boldsymbol{b}) K_i(\boldsymbol{b})$.

5.2. **Two-Level Batching.**

The verifier draws $\delta, \gamma \in \mathbb{K}$ and forms a single random combination of columns and of kernels:

$$\Phi = \sum_{j=1}^{L} \delta^j w_j, \quad W = \sum_{i=1}^{m} \gamma^i K_i, \quad \sigma = \sum_{j,i} \delta^j \gamma^i v_{j,i}.$$

Because $\Phi \cdot W = \sum_{j,i} \delta^j \gamma^i w_j K_i$, the batched claim $\sum_{\boldsymbol{b}} \Phi(\boldsymbol{b}) W(\boldsymbol{b}) = \sigma$ holds if and only if every $v_{j,i}$ is correct (the cross terms are why the *full* matrix is sent, not only the entries the constraint check uses). The combined codeword of Φ is the same δ-combination of the committed column codewords.

5.3. **Interleaved Sumcheck and Folding.**

Definition 1 (Basefold Opening). To prove $\sum_{\boldsymbol{b}} \Phi(\boldsymbol{b}) W(\boldsymbol{b}) = \sigma$, run an n-round degree-2 sumcheck on the product $\Phi \cdot W$. In round t the verifier's challenge λ'_t is used *both* to bind the sumcheck variable and to fold the codeword of Φ one step (Section 3). The intermediate folded oracles $\Phi_1, ..., \Phi_{L'-1}$ are Merkle-committed and their roots absorbed into the transcript; after L' folds the remaining polynomial is small and sent in the clear as its coefficient vector Φ_{final}.

The final sumcheck value must equal $\tilde{\Phi}(\boldsymbol{\lambda}') \cdot \widetilde{W}(\boldsymbol{\lambda}')$; the verifier computes $\widetilde{W}(\boldsymbol{\lambda}')$ itself (all kernels are eq/rotation kernels, evaluable in $O(n)$), and reads $\tilde{\Phi}(\boldsymbol{\lambda}')$ off Φ_{final}. This last check is what makes the claim matrix *binding*.

5.4. **Query Phase.**

Proximity to the code is enforced by repeated queries. The verifier derives q query positions from the transcript; for each, it opens the stage commitments at the position (recovering Φ at x and $-x$ via the paired leaf), then walks the folding cascade, at each level checking the folding-consistency relation against the opened pair and its Merkle path, down to Φ_{final}. A prover that committed a word far from the code is rejected with probability growing in q; the number of queries is chosen for the target soundness at rate ρ.

6. The Basic-Proof Protocol

We now assemble the pieces into the protocol that produces one basic proof, and state what it attests.

6.1. **Transcript Schedule.**

The proof is made non-interactive by Fiat–Shamir [FS87] over a Poseidon2 sponge [GKS23]; the same hash instantiates the Merkle trees. The prover and verifier maintain a common transcript, and every challenge

is squeezed from it after the messages it must depend on have been absorbed. For an $n = \log_2 N$ instance of AIR $\mathcal{A}_\tau$ with σ stages of witness columns:

1. Absorb the statement: the AIR identifier τ, the public inputs $\boldsymbol{p}$, and the fixed/cached commitments from vk.
2. Absorb the stage-1 witness commitment. Absorb the stage challenges (Section 7); for a single instance in isolation these are inputs, not squeezed here.
3. Absorb the remaining stage commitments and the AIR values $\boldsymbol{a}$ (prover messages, bound before any zerocheck randomness).
4. Squeeze the zerocheck randomness $\boldsymbol{r} \in \mathbb{K}^n$ and the constraint-batching challenge $\mu \in \mathbb{K}$.
5. Run the zerocheck: for $t = 1, ..., n$, absorb the round polynomial g_t and squeeze λ_t.
6. Absorb the claim matrix $\left(v_{j,i}\right)$; squeeze the opening batching challenges $\delta, \gamma \in \mathbb{K}$.
7. Run the opening: for each round, absorb the round polynomial and (until the last) the folded-oracle root, squeezing λ'_t; absorb Φ_{final}.
8. Squeeze the q query positions and answer them.

6.2. **The Proof Object.**

Definition 1 (Basic Proof). A *basic proof* π for an instance of $\mathcal{A}_\tau$ consists of: the witness-stage commitment roots and the cached-commitment roots; the AIR values $\boldsymbol{a}$ (including the exposed bus contribution s) and the publics $\boldsymbol{p}$; the zerocheck round polynomials $(g_t)_{t=1}^{n}$; the claim matrix $\left(v_{j,i}\right)$; and the opening (opening round polynomials, folded-oracle roots, Φ_{final}, and the q query openings with their Merkle paths).

6.3. **Verification.**

The verifier $V(\mathsf{vk}, x, \pi)$, with public statement $x = (\boldsymbol{p}, \alpha, s)$, replays the transcript to re-derive every challenge and checks, in order: the claim matrix and round polynomials have the expected shapes; each zerocheck round satisfies $g_t(0) + g_t(1) = s_{t-1}$; the zerocheck final identity $s_n = \mathrm{eq}(\boldsymbol{r}, \boldsymbol{\lambda}) \cdot \sum_i \mu^{i-1} C_{i(\ldots)}$ holds when the constraints are evaluated on the claimed openings; each boundary constraint vanishes on the claimed corner openings; the opening's final identity $\sigma_n = \tilde{\Phi}(\boldsymbol{\lambda}') \cdot \widetilde{W}(\boldsymbol{\lambda}')$ holds; and every query's folding-consistency and Merkle checks pass against the roots.

6.4. **The Relation Attested.**

Collecting these, a basic proof attests exactly the AIR-leaf relation of the whitepaper:

$$\mathcal{R}^{(\tau)}_{\mathrm{AIR}} = \{(\boldsymbol{p}, \alpha, s; (\boldsymbol{T}, \boldsymbol{a})) \mid (\boldsymbol{T}, \boldsymbol{a}) \text{ satisfies } \mathcal{A}_\tau, \text{consistent with } \boldsymbol{p}, \wedge\, s = \mathrm{BusCont}_\alpha(\mathcal{J}_\tau, \boldsymbol{T}, \boldsymbol{a})\}.$$

The first conjunct is enforced by the zerocheck plus the boundary corner checks; consistency with $\boldsymbol{p}$ is a family of first-row boundary constraints; and $s = \mathrm{BusCont}_\alpha(\mathcal{J}_\tau, \boldsymbol{T}, \boldsymbol{a})$ is enforced as ordinary AIR constraints — the LogUp partial-sum recurrence emitted by the standard library (the whitepaper's LogUp reduction)— with the final value $s = S(N)$ exposed as an AIR value. The soundness-critical randomness α (and its derived companion γ) is supplied as a public input; how it is fixed across instances is the subject of the next section.

7. From AIRs to Circuits

A single basic proof certifies one instance in isolation. A circuit $\mathcal{C}$ is proved by a *set* of basic proofs, one per instance, and two circuit-level facts must additionally hold: all instances must use the *same* LogUp challenges, and the bus must balance globally. This section describes how the backend arranges the first and exposes the data for the second. Folding the set into a single recursive proof is the whitepaper's aggregation layer (the whitepaper's Recursion section) and is not yet implemented; what follows is the pre-aggregation construction the current system uses.

7.1. **Why a Common Challenge.**

The LogUp reduction (the whitepaper's LogUp reduction) computes each instance's partial bus sum s_i using challenges α, γ, and global balancing is the scalar identity $\sum_i s_i = 0$. This identity is meaningful only if every s_i is computed under the *same* (α, γ): with per-instance challenges the partial sums live in incommensurable fingerprint spaces and their sum carries no information. A basic proof therefore cannot derive its own challenge from its own transcript; the challenge must be common to the whole set and fixed only after every instance has committed its stage-1 trace.

7.2. **Challenge Derivation.**

Concretely, all instances first commit their stage-1 witnesses. The backend then derives the stage challenges from the entire collection of stage-1 commitment roots and broadcasts them; each basic proof then proceeds with the challenge as a public input, exactly as the whitepaper's leaf proofs assume α rather than derive it. The LogUp challenges α (linear-combination) and γ (fingerprint) are squeezed together from this shared transcript state.

Remark (relation to the whitepaper's LtHash derivation). *The current implementation derives the shared challenge by absorbing the stage-*1 *roots into a Poseidon2 transcript in a fixed instance order and squeezing* $\mathbb{K}$*. This is an order-*dependent *hash: it is sound as a common-challenge mechanism but forces a canonical ordering and is inherently sequential. The whitepaper (its Challenge Derivation section) instead specifies an order-*independent *multiset hash (LtHash), whose homomorphism lets disjoint batches be hashed in parallel and merged, and whose correctness is checked incrementally inside the aggregation tree. The transcript-absorption used here is the pre-aggregation stand-in for that scheme; migrating to LtHash is part of wiring the multilinear backend into recursion.*

7.3. **Set-Level Verification.**

Verifying a circuit reduces, for now, to verifying the complete set of basic proofs together:

1. Re-derive the shared challenges from the stage-1 roots carried by the proofs, and require each proof to have used them (a missing or altered proof changes the derived challenge and the whole set is rejected).
2. Verify each basic proof against its AIR's vk as in Section 6.
3. Aggregate the exposed AIR values across the set according to each bus's combination rule and check the global balancing identity $\sum_i s_i = 0$, i.e. evaluate the circuit's global constraints on the aggregated values.

Steps 1 and 3 are precisely the obligations the whitepaper assigns to the aggregation tree's Final Verifier —checking that the assumed challenge matches what the commitments accumulate to, and that the partial sums balance. Here they are discharged directly over the proof set rather than recursively; the migration to recursion replaces this set-level check with the aggregation tree without changing what each basic proof attests.

Remark (verifying-key binding of committed roots). *A basic proof currently carries the Merkle roots of its fixed and cached (e.g. ROM) commitments and the verifier trusts them as presented, analogously to reading them from* vk*. Binding these roots to an externally fixed verification key — either by recomputing them at verification from the committed data or by emitting them into* vk *at setup— is pending and is the multilinear analogue of the univariate constant-tree/verification-key artifacts.*

8. Instantiation and Scope

We close by recording the concrete instantiation and the current boundaries of the construction.

8.1. **Instantiation.**

The backend is instantiated over Goldilocks $\mathbb{F}$ ($p = 2^{64} - 2^{32} + 1$) with the degree-three extension $\mathbb{K}$ carrying all challenges, sumcheck messages, and folded oracles (about 192 bits, comfortable for the Schwartz–Zippel terms). Merkle commitments and the Fiat–Shamir transcript use Poseidon2 [GKS23] with arity-4 trees. Reed–Solomon encoding uses a coset of the Goldilocks two-adic subgroup. The default parameters are log-blowup $\kappa = 2$ (rate $\rho = 1/4$) and $q = 50$ queries; these are conjectured, not analysed, soundness settings, with no proof-of-work grinding yet.

Extension-valued columns (the stage-≥ 2 LogUp accumulators, whose values lie in $\mathbb{K}$) are committed as three base-field columns and reassembled by the constraint evaluator; the commitment layer is oblivious to this. Cached commitments enter as additional fixed matrices, one Merkle root each, bound into the transcript alongside the fixed columns.

8.2. **Supported Today.**

The construction proves multi-stage AIRs with the full standard-library argument set —permutation, lookup, range-check, and copy arguments, hence LogUp buses— together with transcript challenges, AIR and airgroup values, and stage-0 cached commitments (ROMs). It has been exercised end to end both on hand-built AIRs and on ZisK circuit components, producing and verifying complete basic-proof sets with the shared-challenge and global-balancing checks of Section 7.

8.3. **Not Yet Covered.**

The following are deliberately out of scope for this first version and are tracked for later work:

- **Recursion.** There is no aggregation: the backend emits base proofs and verifies them as a set. Folding a set into a single proof, migrating challenge derivation to the LtHash multiset hash, and binding committed roots into vk are the recursion-integration tasks.
- **Remaining frontend features.** Proof values, frame boundaries spanning arbitrary row ranges, and boundary constraints referencing shifted columns are not handled; these are additions to the constraint compiler rather than to the protocol.
- **Soundness accounting.** The query count and rate are placeholders; a formal choice (and optional grinding) requires the list-decoding analysis of the Basefold/WHIR line [Arn+24, ZCF23] and is not yet carried out.
- **Performance.** The prover is a scalar CPU reference implementation. The commitment encoding, sumcheck rounds, and folding are all parallelisable, and the whole backend is a candidate for the same GPU acceleration as the univariate prover; none of that is done here.

8.4. **Outlook.**

The multilinear backend reproduces, with a different cryptographic core, exactly the basic-proof obligation the ZisK whitepaper assigns to AIR leaves. Its distinguishing properties —no quotient, constraint degree paid in field work rather than commitment size, and shifted/boundary accesses reduced to kernel evaluations against the base columns— make it a natural fit for the arithmetization framework, and its proof object is already shaped for the aggregation layer that will consume it.

References

Ben+18. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable, Transparent, and Post-Quantum Secure Computational Integrity, https://eprint.iacr.org/2018/046

Lun+92. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic Methods for Interactive Proof Systems. In: Journal of the ACM. pp. 859–868 (1992)

ZCF23. Zeilberger, H., Chen, B., Fisch, B.: BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes, https://eprint.iacr.org/2023/1705

Che+22. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates, https://eprint.iacr.org/2022/1355

Arn+24. Arnon, G., Chiesa, A., Fenzi, G., Yogev, E.: WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification, https://eprint.iacr.org/2024/1586

FS87. Fiat, A., Shamir, A.: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In: Advances in Cryptology — CRYPTO '86. pp. 186–194. Springer (1987)

Gru24. Gruen, A.: Some Improvements for the PIOP for ZeroCheck, https://eprint.iacr.org/2024/108

GKS23. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A Faster Version of the Poseidon Hash Function. In: Progress in Cryptology — AFRICACRYPT 2023. pp. 177–203. Springer (2023)